

Integrating a LangChain Project with LangSmith

Five steps — tracing is configuration, not code

LangChain emits traces automatically once the environment variables are set. You do not instrument your chains or agents by hand; every chain, agent, tool call and LLM invocation is captured as soon as tracing is switched on.

Step 1 — Get an API key

Sign up at smith.langchain.com, then go to **Settings** → **API Keys** → **Create API Key**. Keys begin with `lsv2_pt_`. Treat it as a secret — it grants write access to your traces.

Step 2 — Install

```
pip install -U langsmith langchain
```

`langsmith` already ships as a dependency of `langchain`, but pin it explicitly so upgrades are deliberate rather than incidental.

Step 3 — Set the environment variables

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY="lsv2_pt_..."
export LANGSMITH_ENDPOINT="https://api.smith.langchain.com"
export LANGSMITH_PROJECT="weather-agent" # created automatically on first run
```

VARIABLE	PURPOSE	STATUS
<code>LANGSMITH_TRACING</code>	Master switch. Must be <code>true</code> .	Required
<code>LANGSMITH_API_KEY</code>	Your key from the LangSmith UI (<code>lsv2_pt_...</code>).	Required
<code>LANGSMITH_ENDPOINT</code>	Regional API URL. Defaults to US.	Optional
<code>LANGSMITH_PROJECT</code>	Groups runs. Created on first trace if absent.	Optional

Or as a `.env` file, loaded before any LangChain import:

```
from dotenv import load_dotenv
load_dotenv() # must run BEFORE importing langchain modules

from langchain.agents import create_agent
```

Regional endpoints — use the one matching your organisation:

US (default)	https://api.smith.langchain.com
EU	https://eu.api.smith.langchain.com
APAC	https://apac.api.smith.langchain.com
AWS US	https://aws.api.smith.langchain.com

Step 4 — Run your existing code, unchanged

No edits are required to your application:

```
result = agent.invoke({"messages": [{"role": "user", "content": "Weather in Hyderabad?"}]})
```

Open the project in the LangSmith UI and you will see the full tree: agent → tool call → tool arguments → LLM tokens and latency, with timing at every node.

Step 5 — Verify it is actually on

```
import os
from langsmith import Client

print("tracing:", os.getenv("LANGSMITH_TRACING"))
print("projects:", [p.name for p in Client().list_projects(limit=5)])
```

If this raises an authentication error, the key is wrong. If it prints successfully but no traces appear in the UI, `LANGSMITH_TRACING` was not set before the LangChain modules were imported.

Two gotchas

The old variable names still work but are deprecated. `LANGCHAIN_TRACING_V2`, `LANGCHAIN_API_KEY` and `LANGCHAIN_PROJECT` map to the `LANGSMITH_*` names. Do not set both prefixes — mixed configuration produces confusing partial results.

Tracing is fire-and-forget and batched in the background. A short-lived script can exit before the buffer flushes. Call `langsmith.flush()` (or `Client().flush()`) at the end of a script or serverless handler, or runs will go missing without any error.